

Spark Connect vs. Spark Classic

Wenn eine Anwendung mit Spark kommuniziert, kann sie das auf zwei verschiedene Arten tun: über die klassische Architektur ("Spark Classic") oder über "Spark Connect". Dieses Kapitel erklärt den Unterschied und zeigt typische Stolperfallen, die beim Umstieg auf Spark Connect auftreten können.

Was ist Spark Connect?

Spark Connect ist ein gRPC-basiertes Protokoll innerhalb von Apache Spark. Es definiert, wie eine Client-Anwendung mit einem entfernten ("remote") Spark-Server kommunizieren kann. Statt dass der Client direkt Teil des Spark-Treiber-Prozesses ist, schickt er seine Befehle über das Netzwerk an einen Server, der sie ausführt. Das ermöglicht eine Remote-Ausführung von Spark-Workloads über die DataFrame-API.

Auf Databricks kommt Spark Connect in folgenden Bereichen zum Einsatz:

- Scala- und Python-Notebooks (ab Databricks Runtime 13.3 bzw. 14.3)
- Serverless Compute
- Databricks Connect (Verbindung von externen IDEs zu Databricks)
- Lakeflow-Pipelines

Gemeinsamkeiten: Beide nutzen Lazy Execution

Sowohl Spark Classic als auch Spark Connect arbeiten grundsätzlich nach dem gleichen Prinzip der "verzögerten Ausführung":

Operationstyp	Beispiel	Verhalten
Transformationen	<code>filter</code> , <code>select</code> , <code>limit</code>	Lazy (verzögert) in beiden Varianten
Aktionen	<code>collect</code> , <code>show</code>	Eager (sofort) in beiden Varianten
SQL-Befehle	<code>insert</code> , <code>create</code>	Eager (sofort) in beiden Varianten

Der entscheidende Unterschied: Wann wird das Schema geprüft?

Der wichtigste Unterschied liegt darin, wann Spark das Schema einer Abfrage analysiert (also z. B. prüft, ob eine Spalte existiert oder welchen Datentyp sie hat):

Aspekt	Spark Classic	Spark Connect
Transformationen	Eager – Analyse sofort	Lazy – Analyse erst bei Bedarf
Schema-Zugriff	Lokal berechnet	Per RPC (Netzwerk-Aufruf) mit Caching
Temporäre Views	Im Ausführungsplan fest eingebettet	Nur als Namens-Referenz gespeichert
UDF-Serialisierung	Bei der Erstellung der Funktion	Erst bei der Ausführung

Praktische Konsequenz: Spark Classic analysiert Abfragepläne sofort und erkennt viele Fehler (z. B. eine falsch geschriebene Spalte) schon beim Erstellen des DataFrames. Spark Connect dagegen schickt zunächst einen noch nicht aufgelösten Plan zum Server – Fehler tauchen deshalb oft erst später auf, nämlich wenn das Schema abgefragt wird oder eine Aktion ausgeführt wird.

Best Practices für die Arbeit mit Spark Connect

1. Eindeutige Namen für temporäre Views verwenden

Weil ein DataFrame in Spark Connect nur den *Namen* einer temporären View speichert (nicht deren Inhalt), kann es zu unerwarteten Ergebnissen kommen, wenn eine View später mit gleichem Namen überschrieben wird. Empfehlung: eindeutige Namen z. B. mit einer UUID erzeugen:

```
temp_view_name = f"`temp_view_{uuid.uuid4()}`"
```

2. UDFs bewusst mit Closures kapseln

Da Spark Connect benutzerdefinierte Funktionen (UDFs) erst bei der Ausführung serialisiert (nicht bei der Definition), können sich referenzierte externe Variablen bis dahin ändern. Ein Beispiel für unerwartetes Verhalten:

```
x = 123
@udf("INT")
def foo():
    return x
x = 456
df.show() # Gibt 456 statt der erwarteten 123 aus
```

Lösung: Die Erzeugung der UDF in eine Hilfsfunktion kapseln, damit der Wert von  zum gewünschten Zeitpunkt gebunden wird ("early binding").

3. Fehler frühzeitig durch gezielte Schema-Abfragen erkennen

Wer Fehler nicht erst spät im Programm entdecken möchte, kann gezielt eine Schema-Abfrage (z. B. `df.columns` oder `df.schema`) auslösen, um die Analyse und damit auch mögliche Fehler frühzeitig zu erzwingen:

```
try:
    df = df.filter(col("non_existent") > 6)
    df.columns # löst Analyse aus und wirft ggf. einen Fehler
except Exception as e:
    print(f"Error: {repr(e)}")
```

4. Häufige Schema-Abfragen vermeiden (Performance)

Jede Abfrage des Schemas (z. B. `df.columns`) verursacht bei Spark Connect einen Netzwerk-Aufruf (RPC) zum Server. Wird das in einer Schleife wiederholt gemacht, kann das die Performance spürbar verschlechtern:

```
# Ungünstig: RPC bei jeder Iteration
for i in range(200):
    if new_col not in df.columns:
        df = df.withColumn(new_col, ...)
```

Besser ist es, die Spaltennamen lokal in einem Set zu pflegen, statt sie wiederholt vom Server abzufragen. Ebenso empfiehlt es sich, Felder direkt aus einem bereits vorhandenen `StructType` - Schema zu extrahieren, anstatt dafür unnötige Zwischen-DataFrames zu erzeugen.

Fazit

Die größte Umstellung beim Wechsel von Spark Classic zu Spark Connect ist die sogenannte "deferred analysis": Das Schema wird nicht mehr sofort lokal berechnet, sondern erst bei Bedarf vom Server geliefert. Wer das versteht und bei temporären Views, UDFs und Schema-Zugriffen bewusst vorgeht, vermeidet die typischen Fehler und Performance-Probleme.

Quelle: <https://docs.databricks.com/aws/en/spark/connect-vs-classic>
Stand: 2026-08-06